Project 2: Modelling daily cycling demand in Edinburgh

Group members: Alice Miller, Saba Sitter, Jake Saunders

Word count: [XXX words]

1 Introduction

The "cycle daily" dataset consists of approximately 2,180 daily observations ranging from 2020 to 2025, sourced from Edinburgh's network of automatic cycling counters, along with local weather and temporal data. This data comes from sensors at specific locations across the city, so it's possible not every cyclist is accounted for. Similarly, some cyclists may pass through multiple sensors in a day and be observed more than once. However, they are a good indicator of how cycling demand varies across different days and weather patterns. Edinburgh relies heavily on public transport for its residents and tourists to navigate the city. For future funding allocation and planning, it is important to know how many cyclists are on the road in order to evaluate whether the infrastructure (cycle lanes, crossings etc) and investment in schemes (eg rentable Voi bikes) need to adapt to meet demand. The modelling goal is to develop a sequence of increasingly complex linear regression models to isolate how temporal and weather variables, such as the day of the week and the temperature, influence cycling demand. Ultimately, the goal is to extrapolate cycling demand so that Edinburgh City Council and Transport Scotland can optimise infrastructure investment over the next decade to align with their active travel priorities.

2 Exploratory data analysis

To begin investigating the impact of weather on the number of daily cyclists, we begun with some exploratory data analysis. This involved converting month to an ordered factor, redefining the day of week predictor with explicit levels. Additionally, we created a new variable 'trend' which symbolises the number of days since the first observation (01/01/2020), along wtih binary indicators 'is_covid' and 'is_holiday' which represent the UK Covid lockdown (23/03/2020 - 31/03/2021) and the yearly festive period (24th - 31st December) respectively.

We initially found the summary statistics for the number of daily cyclists and the weather data.

Table 1: Summary Statistics for count, temp mean, temp min, temp max

Variable	Mean	SD	Min	Max
count	10815.47	4728.88	1264.00	43864.00
temp_mean	9.69	4.79	-8.08	22.96
temp_min	5.92	4.98	-14.00	18.00
temp_max	13.15	5.31	-3.00	31.00

We can see that the number of daily cyclists varies from $1,264$ to $43,864$, a variation of $\approx 3370\%$. Intuitively, one would think that a warmer average weather would lead to more cyclists.

We then plotted a time series of the daily cyclist count over the period of observations.

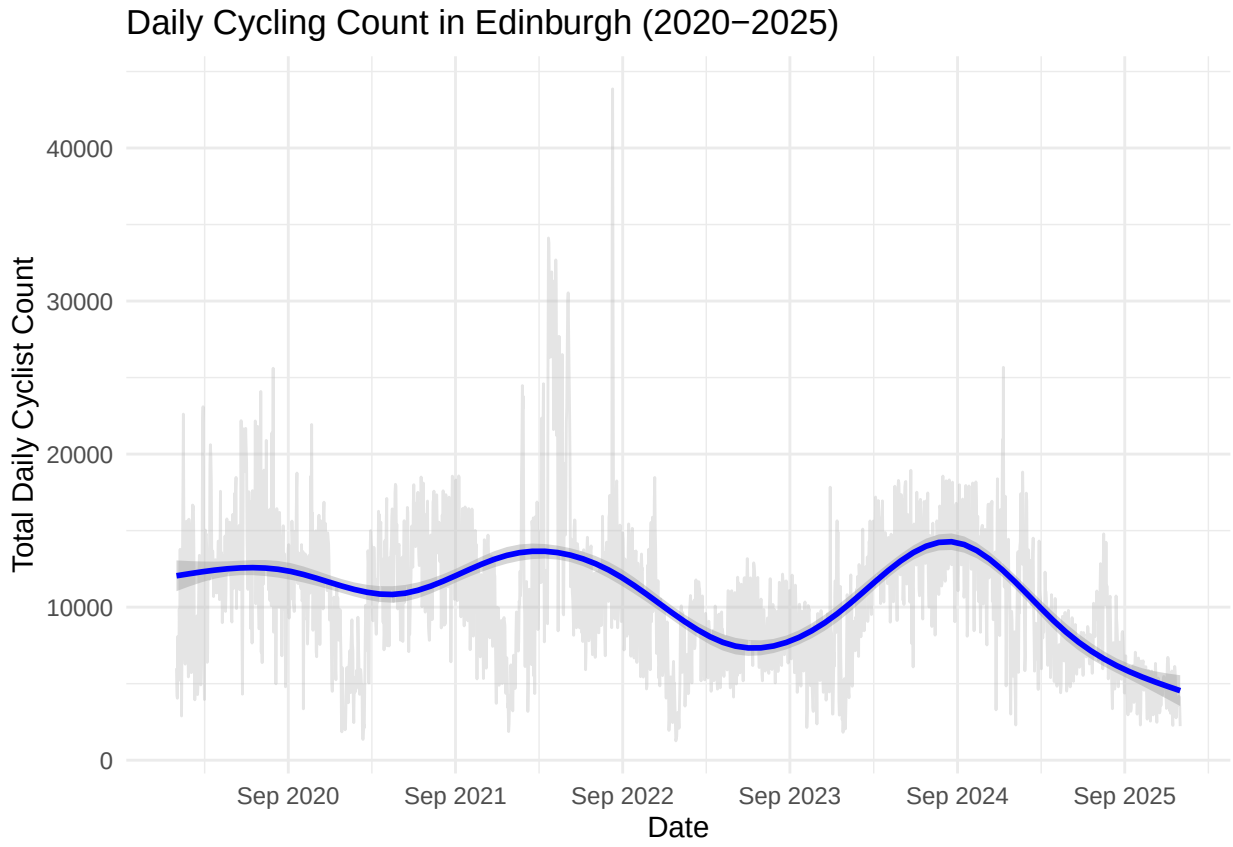


Figure 1: Time series of daily cyclist counts 2020-2025

We can see from Figure X that there are yearly fluctuations, with peaks and troughs occurring seasonally. Overall, the cyclist count seems to be decreasing over time. There is a visible peak on and surrounding the 10th of August 2022, which we investigated. We found that there was a London-Edinburgh-London bicycle event taking place over this period with over 2000 riders[1]. Supposing each cyclist passed through multiple sensors over the course of the days, this likely explains the inflated numbers.

We then created boxplots of the daily cyclist count by month and by the day of week to observe whether they had any significant impact on the number of daily cyclists.

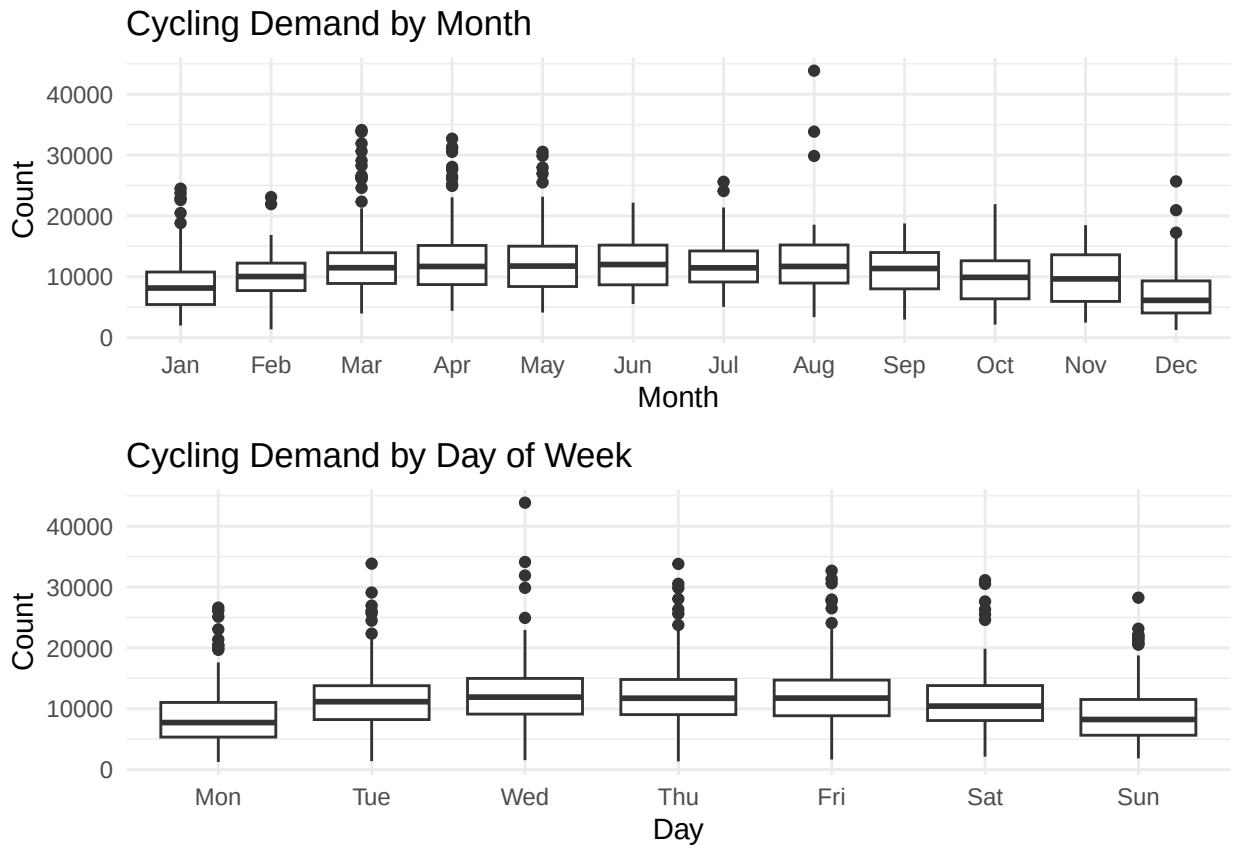


Figure 2: Boxplots of cyclist count by month and day of week

We can see from Figure 2 that the colder months (October - February) have slightly lower cyclist counts, with the other months remaining relatively constant. There are a lot of outliers, especially in Spring. Observations with unusually high cyclist counts could be attributed to special events such as races, bank holidays, and exceptionally sunny days. Outside of winter, the month doesn't appear to have a tremendous impact on the median daily cyclist count.

The second boxplot indicates that there tends to be less cyclists on Mondays and Sundays, with the other days appearing relatively constant.

We then plotted the daily cyclist count against the mean temperature, with red and blue lines indicating regression and quadratic relationship respectively.

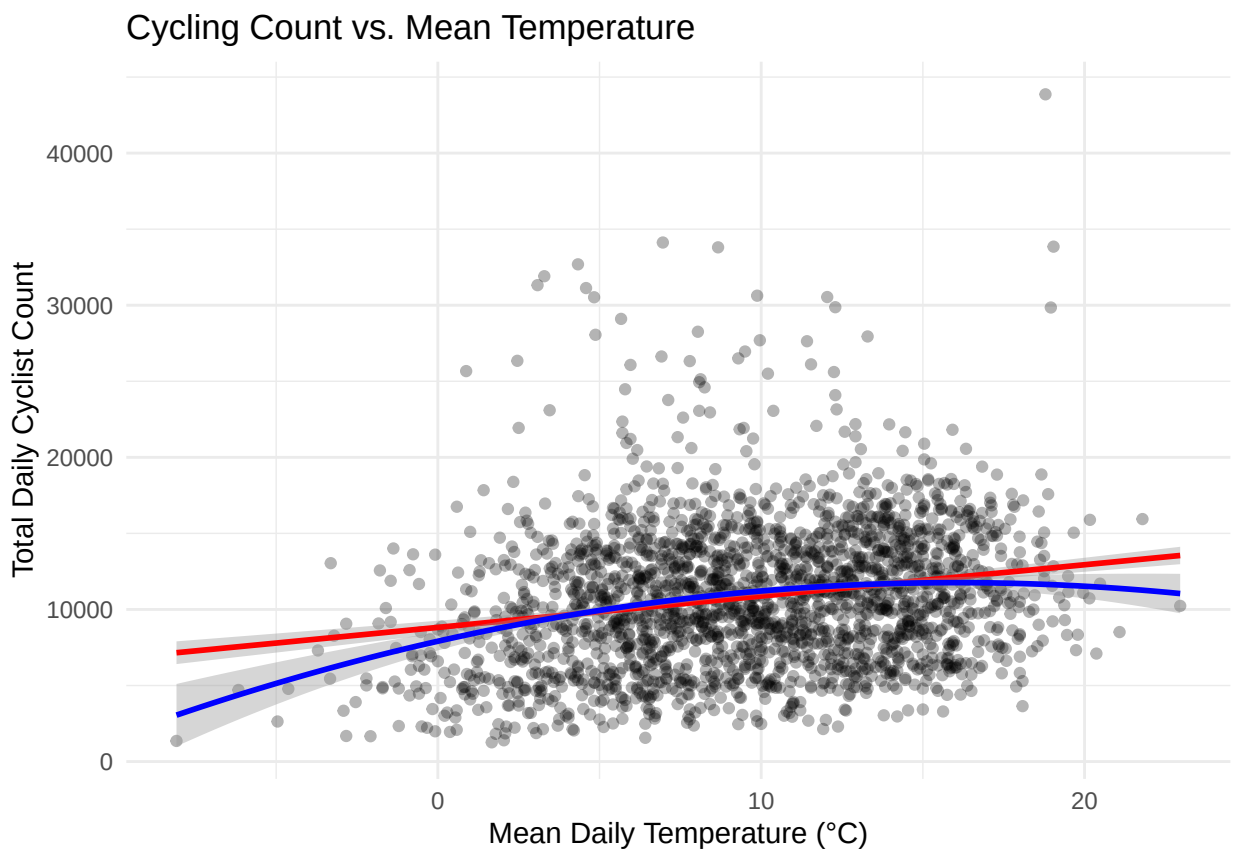


Figure 3: Daily cyclist count vs mean temperature

There is a visible positive correlation between temperature mean and daily cyclist count, which was to be expected. There are less observations for the days with the lowest and highest mean temperatures, so there is not a clear trend that we can see for those days. There is a large variation in cyclist counts for days with the same temperature, so it is not feasible to predict counts based off of mean temperature alone.

We noticed dips in cyclist numbers between Christmas Eve and New Years Eve, presumably due to less commuters on the road. We created the following graph to illustrate this over the course of the 5 years.

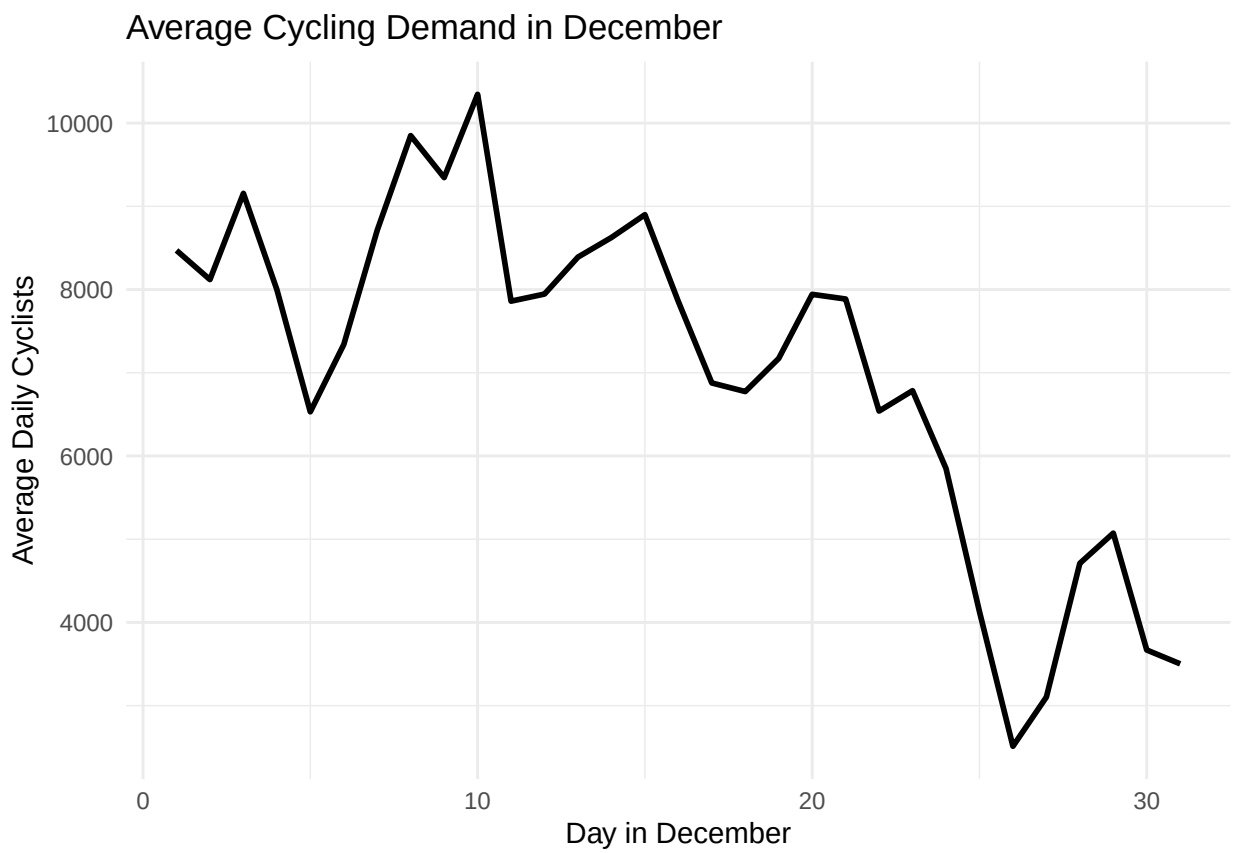


Figure 4: Line Graph of average daily cyclists in decemeber

Clearly, there is a substantial decrease in cyclist numbers between the specified days. This led us to add whether a day falls between the 24th and 31st of December in our final model as 'is_holiday'.

We also created a chart that demonstrates the effect of whether a day is lockdown on daily cyclist counts.

3 Baseline model

We fit an initial model, M0, that uses the mean temperature, along with the month and whether it is a weekend in order to predict the daily cyclist count.

The equation for M0 is:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp_mean}_i + \beta_2 \text{weekend}_i + \beta_3 \text{month}_i + \epsilon_i$$
$$\epsilon_i \sim N(0, \sigma^2)$$

The following table shows the summary statistics for M0:

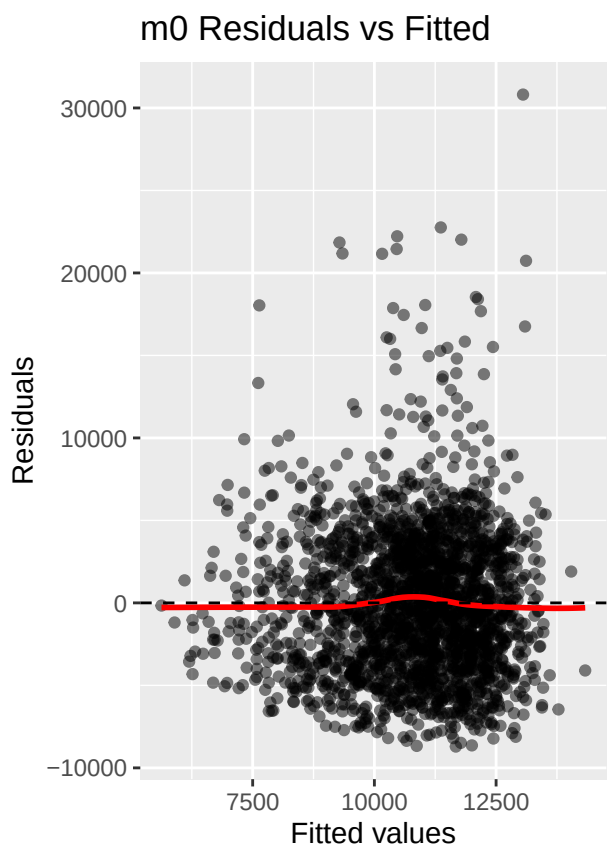
Table 2: Summary of M0

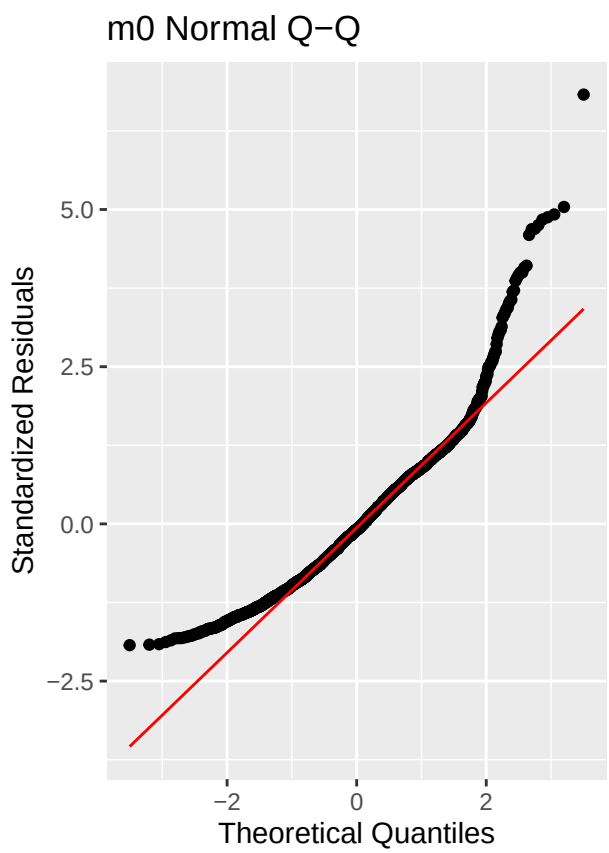
	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	10385.8188	267.01547	38.895944	0

	Estimate	Std. Error	t value	Pr(>|t|)
temp_mean	247.1791	20.76877	11.901478	0
as.numeric(weekend)	-1246.8883	214.17220	-5.821896	0
as.numeric(month)	-247.1840	28.80970	-8.579887	0

For every 1 degree increase in the mean daily temperature, the model predicts an increase of 247 cyclists per day, and predicts a 1247 decrease in cyclist count on weekends compared to weekdays. Since M0 uses month as a numeric predictor, it ignores the fact that cyclist counts generally increase in summer and fall in winter. Currently, the model suggests that there is a decrease of 247 cyclists for each month passing.

We then assessed the diagnostic plots for M0.





The Normal

Model 0 (m0) shows a mean variance of 0, independent of the fitted values. The residuals show a non-linear pattern, with an arguable 'n' shape above the mean line, indicating there is an underlying non-linear structure yet to be included in our model.

The QQ plot demonstrates right skewing and heavy tail in the positive quantiles - thus the normality assumption for linear regression is violated. This indicates the model is particularly bad at predicting high count observations.

4 Model M1

We created a further model, M1, which adds in the trend variable as a predictor to capture the drift in cycling levels over 2020-2025.

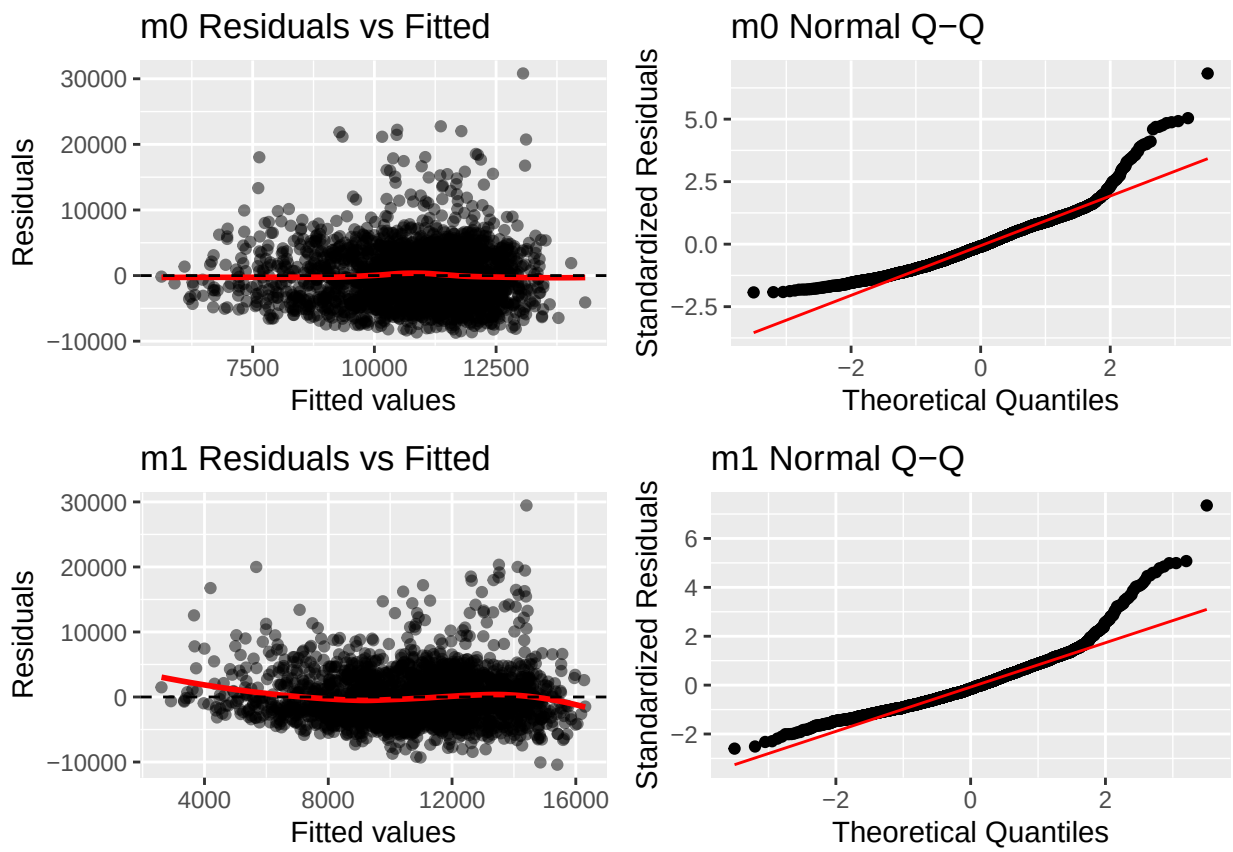
Our proposed equation for M1 (identifiable form) is:

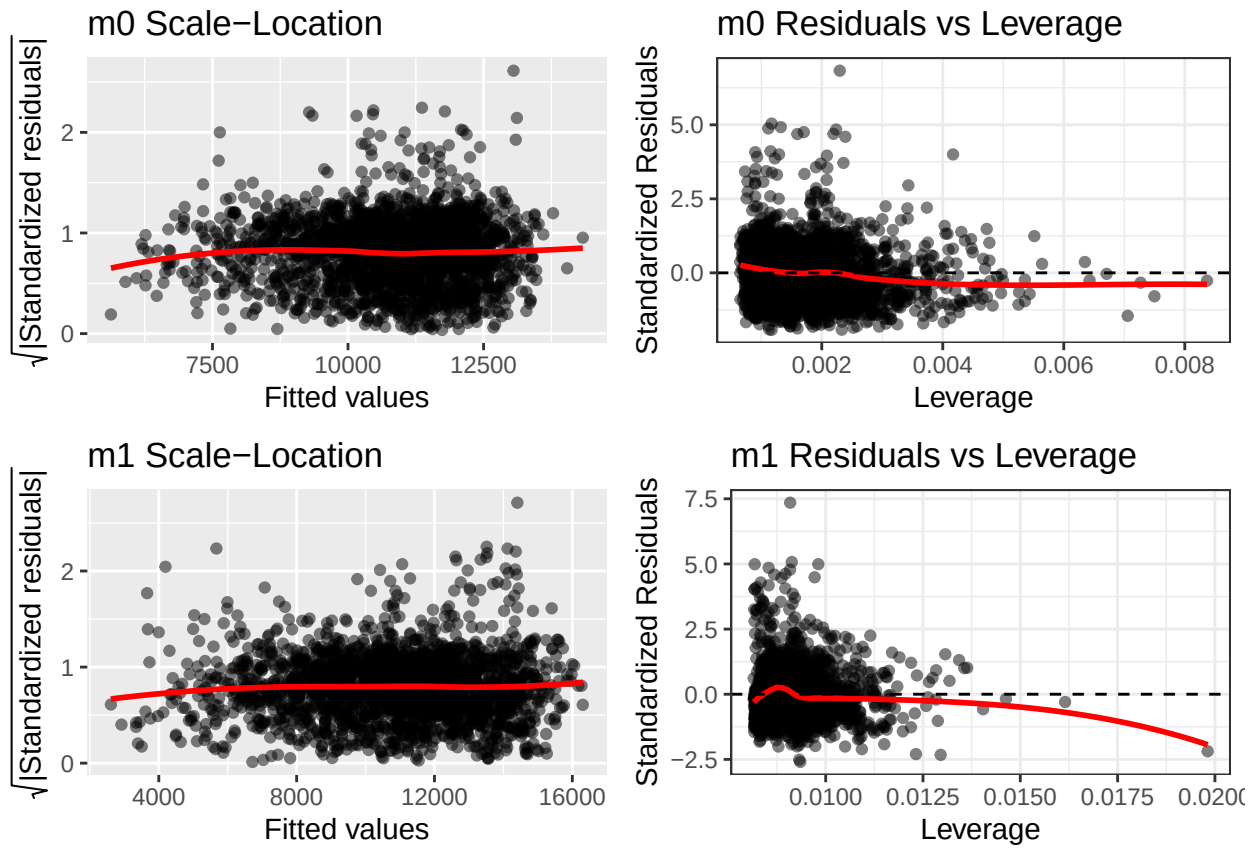
$$\text{count}_i = \beta_0 + \beta_1 \text{temp_mean}_i + \beta_2 \text{trend}_i + \sum_{j=2}^{12} \gamma_j \mathbb{I}(\text{month}_i = j) + \sum_{k=2}^{7} \delta_k \mathbb{I}(\text{dow}_i = k) + \epsilon_i$$

$$\epsilon_i \sim N(0, \sigma^2)$$

Day 1 (Monday) and month 1 (Jan) are encapsulated in the intercept term β_0 Every other month or day is measured as the expected difference in count from the baseline.

We investigate m1 over m0 because m1 has the granularity for the effect of different weekdays rather than simplistic weekend or non weekend effects. Changing 'month' to a factor variable allows for building a seasonal profile rather than assuming the month index (1-12) is linear with the count. We omit the weekend term and numerical version of 'month' to avoid collinearity with their 'factor' versions and because the covariate 'dow' is collinear with 'weekend'.





Plot grid 1: Observe in the bottom left, the mean residual line follows a slight sigmoid shape, suggesting the linear assumption is still violated for model 1. On a positive note, the 'n' pattern about the mean is less pronounced for model 1. The bottom right Q-Q plot looks slightly more aligned to the normal line at the lower tail (compared to M0). This suggests the prediction for the low count days is better.

Plot grid 2: Observe the leverage and thus influence is larger for the outliers of m1.

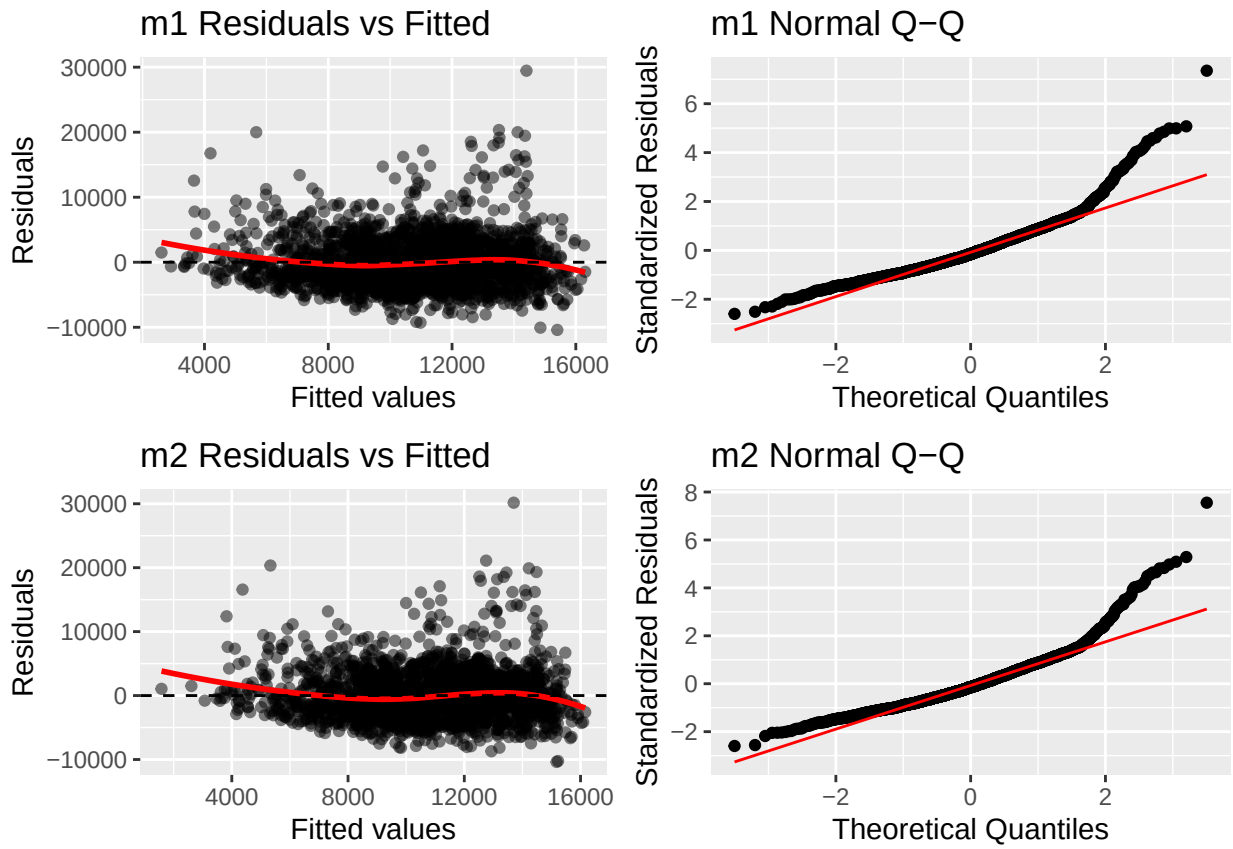
4.1 Model M2: Nonlinear temperature

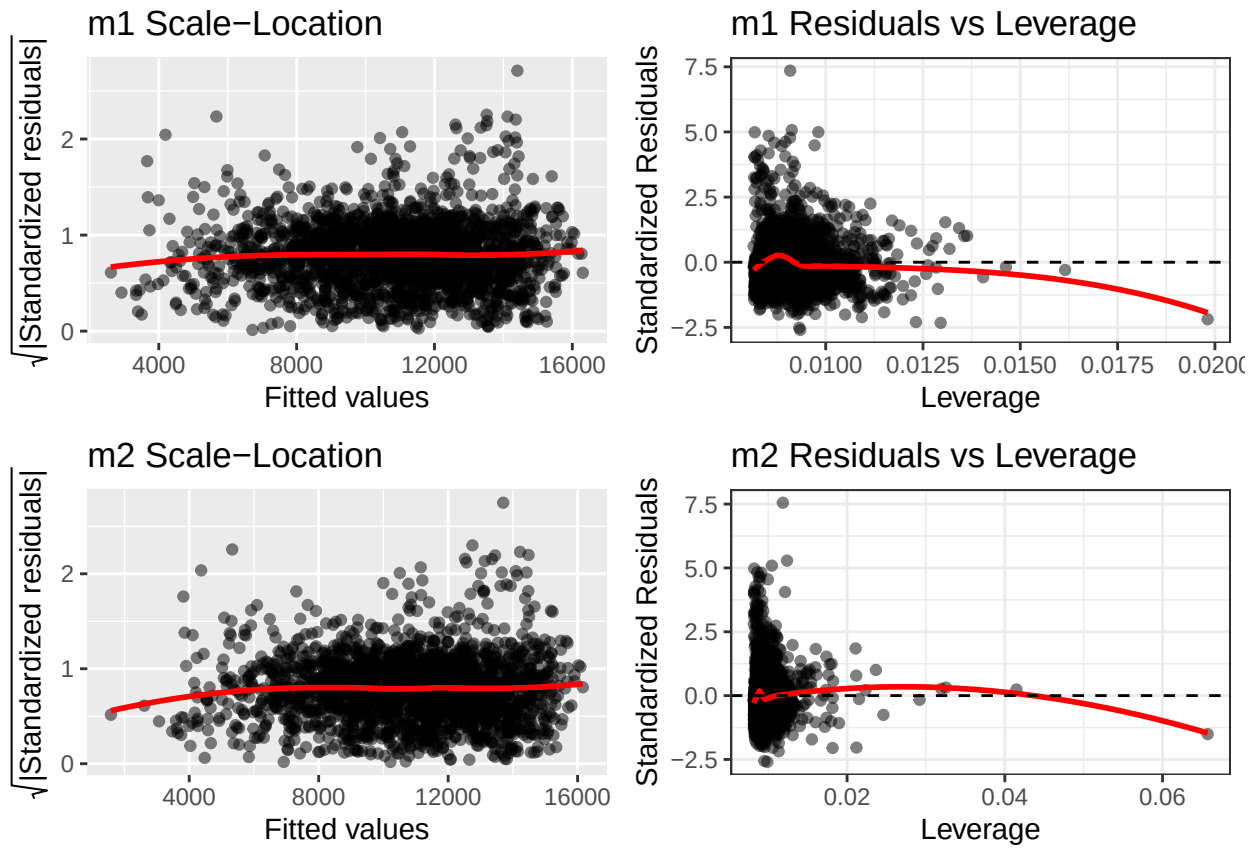
Our proposed equation for M2 (identifiable form) is:

$$\text{count}_i = \beta_0 + \beta_1 \text{temp_mean}_i + \beta_2 \text{temp_mean}_i^2 + \beta_3 \text{trend}_i + \sum_{j=2}^{12} \gamma_j \mathbb{I}(\text{month}_i = j) + \sum_{k=2}^{7} \delta_k \mathbb{I}(\text{dow}_i = k) + \epsilon_i$$

$$\epsilon_i \sim N(0, \sigma^2)$$

From the temperature vs count scatter plot, we see there is an underlying some non linear relationship as we can see some curvature. Hence, if we add the quadratic 'mean temperature' term to model m1, the underlying structure maybe better covered by the model:


m1 Residuals vs Fitted
Residuals
Fitted values
30000
20000
10000
0
−10000
4000
8000
12000
16000
m1 Normal Q−Q
Standardized Residuals
Theoretical Quantiles
6
4
2
0
−2
m2 Residuals vs Fitted
Residuals
Fitted values
m2 Normal Q−Q
Standardized Residuals
Theoretical Quantiles
8



M2 shows a general alignment with linear modelling assumptions since there are no notable violations. Observe both models are almost identical from the diagnostic plots, therefore further comparison may be undertaken using the scores. The only noticeable change comes from the leverage of the outliers increasing slightly.

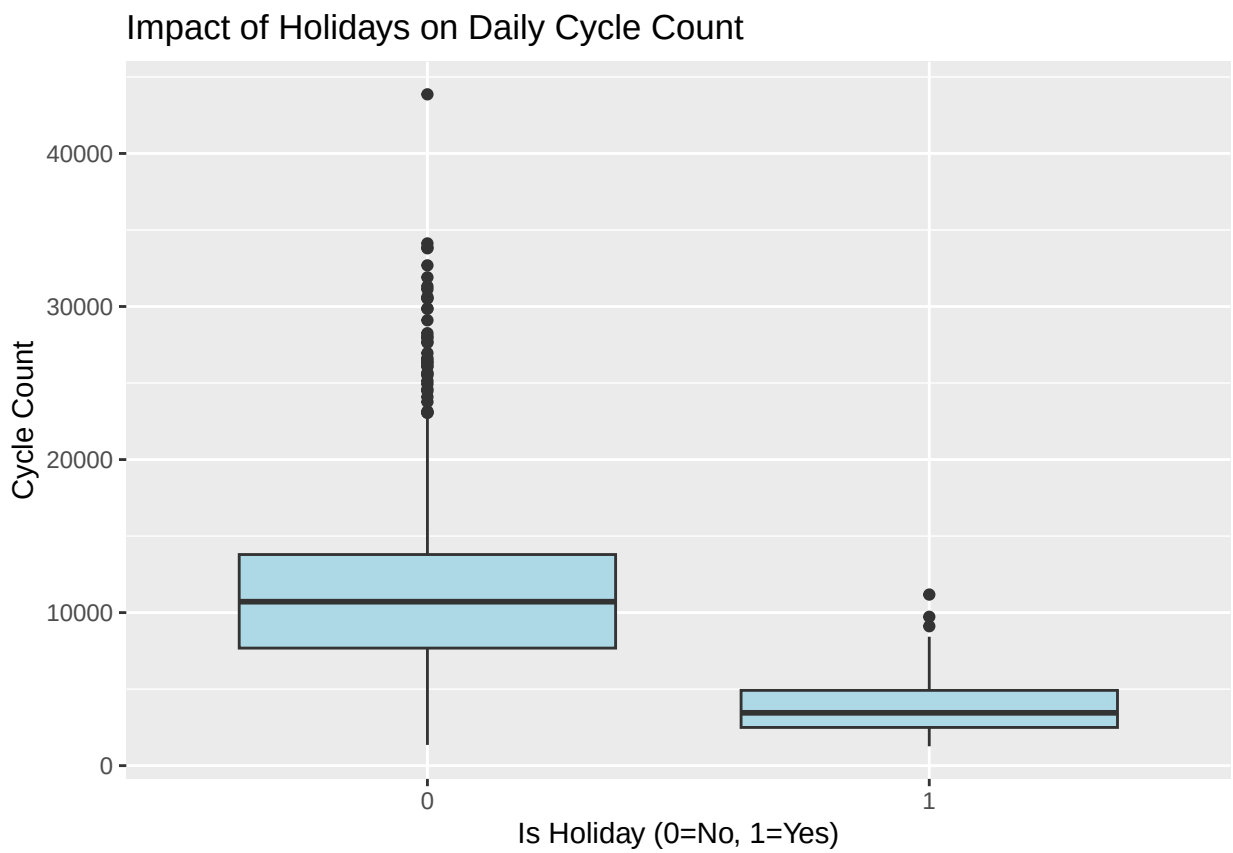
4.2 Model M3: [your extension title]

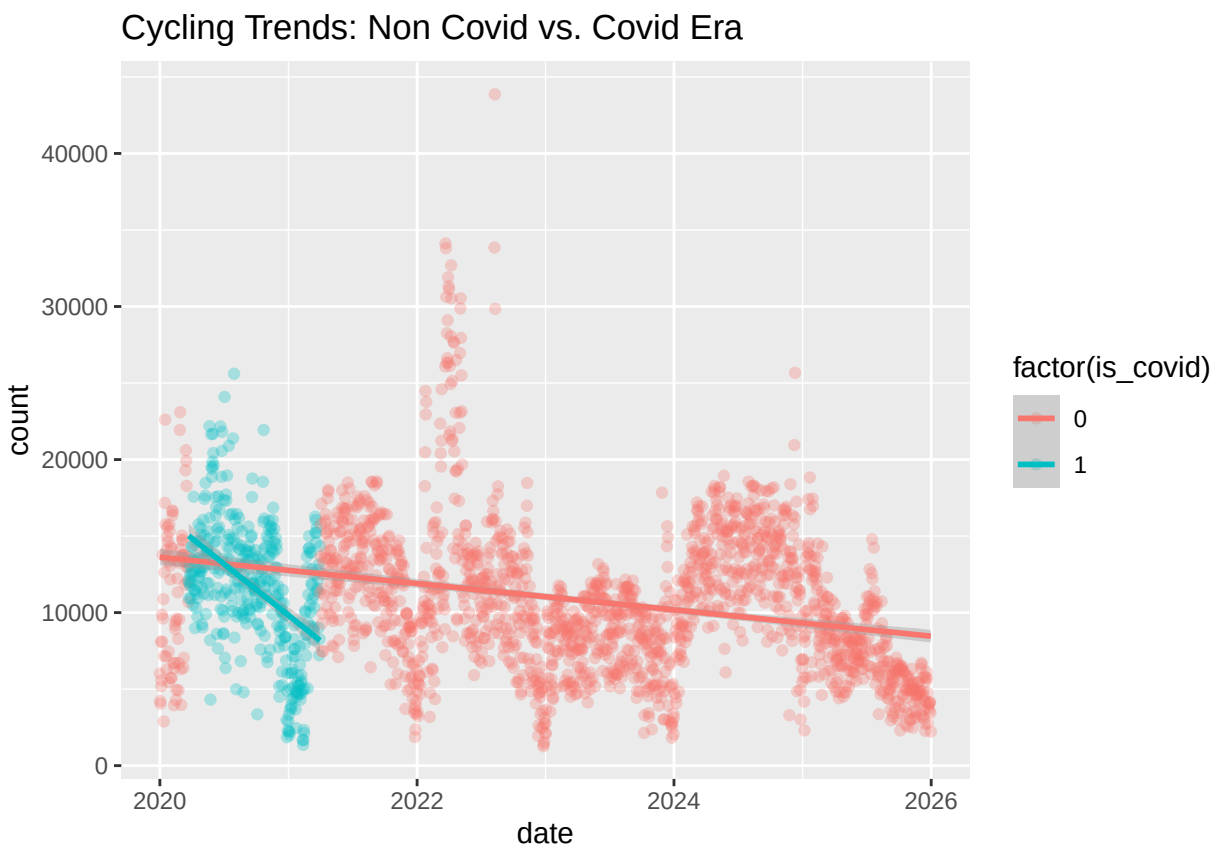
The expression is:

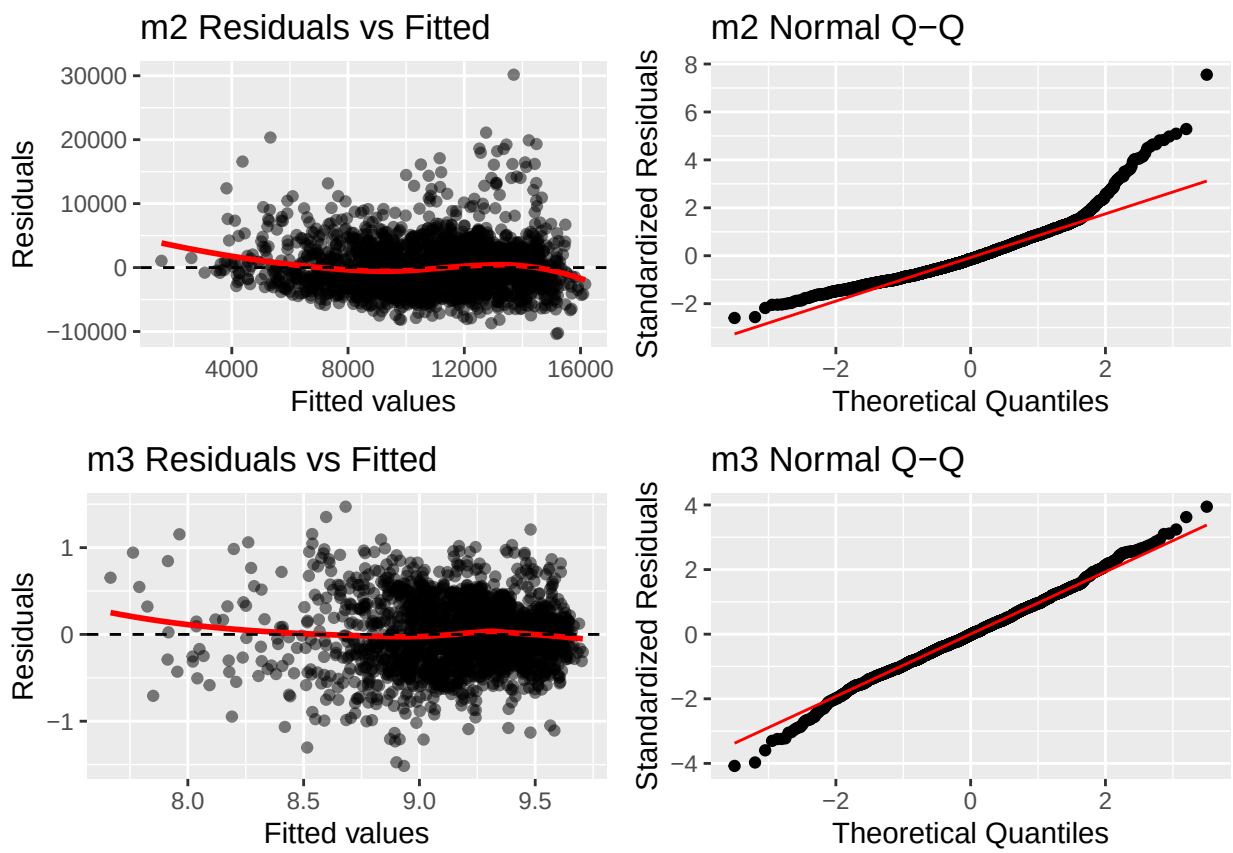
$$\log(count_i+1) = \beta_0+\beta_1 temp_mean_i+\beta_2 temp_mean_i^2+\beta_3 trend_i+\sum_{j=2}^{12}\gamma_j \mathbb{1}(month_i = j)+\sum_{k=2}^{7}\delta_k \mathbb{1}(dow_i = k)+\beta_4 is_covid_i+\beta_5 i$$

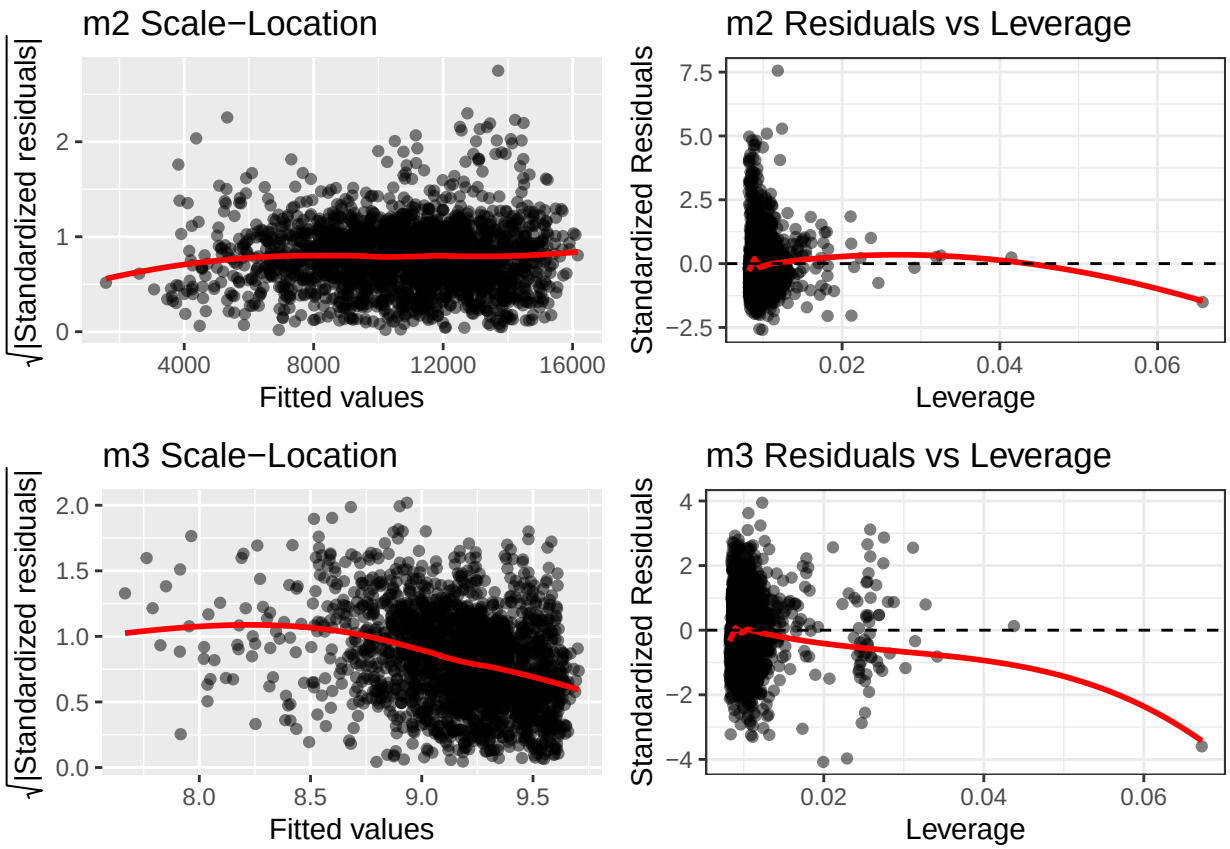
$$\epsilon_i \sim N(0, \sigma^2)$$

To mediate the heavy right tailed QQ plots seen in (m0-2), we apply the log transform to the count response. We addd terms is_covid and is_summer ???


Impact of Holidays on Daily Cycle Count
Cycle Count
40000
30000
20000
10000
0
0
1
Is Holiday (0=No, 1=Yes)


Cycling Trends: Non Covid vs. Covid Era
count
40000
30000
20000
10000
0
2020
2022
2024
2026
date
factor(is_covid)
0
1


m2 Residuals vs Fitted
Residuals
Fitted values
30000
20000
10000
0
−10000
4000
8000
12000
16000
m2 Normal Q−Q
Standardized Residuals
Theoretical Quantiles
8
6
4
2
0
−2
−2
0
2
m3 Residuals vs Fitted
Residuals
Fitted values
1
0
−1
8.0
8.5
9.0
9.5
m3 Normal Q−Q
Standardized Residuals
Theoretical Quantiles
4
2
0
−2
−4
−2
0
2



5 Evaluation

5.1 Statistical Performance

Table of the Leave-one-year-out CV scores.

Table 3: Table of the Leave-one-year-out CV scores

Model	RMSE	MAE	DS	IS
m0	4740.88	3805.56	18.05	25630.46
m1	4587.61	3650.23	18.07	25277.87
m2	4587.61	3650.23	18.07	25277.87
m3	4847.43	3917.76	19.06	34538.19

Table of the RMSE and DS by month

Table 4: Table of the RMSE and DS by months

Month	RMSE	DS
Jan	4141.9	18.25
Feb	3518.0	17.53
Mar	5126.4	18.10

Month	RMSE	DS
Apr	5437.0	18.43
May	4372.4	17.79
Jun	3162.2	17.32
Jul	3118.6	17.19
Aug	4408.1	17.71
Sep	3182.2	17.31
Oct	3357.9	17.47
Nov	3512.2	17.59
Dec	3574.1	18.64

5.2 Practical Performance

5.2.1 5.1 Temperature effects and nonlinearlity

The table shows the different score values for model 2, but using 3 different temperature variables:temp mean, temp min and temp max.

Table 5: Model M2 fitted with daily temp mean, temp min, temp max.

Model	RMSE	MAE	DS	IS
Temp Mean	5459.30	4202.82	18.46	31740.12
Temp Min	5475.41	4216.60	18.48	31929.26
Temp Max	5453.76	4202.23	18.46	31560.07

The model with temp max in the nonlinear term has the best predictive performance because it has the lowest value for each score.

At 5° c,a 1° c increase is associated with approximately 108 additional cyclists.

At 15° c,a 1° c increase is associated with approximately 166 additional cyclists.

We can see the effects a 1° increase has in Spring and winter specifically.

On a Spring day at 5° c,a 1° c increase is associated with approximately 343 additional cyclists.

On a Spring day at 15° c,a 1° c increase is associated with approximately 424 additional cyclists.

On a Winters day at 5° c,a 1° c increase is associated with approximately 56 additional cyclists.

On a Winters day at 15° c,a 1° c increase is associated with approximately 137 additional cyclists.

As we can see, there is huge variablity between seasons. For transport planning this implies that looking at the range of temperatures for the day is more importantas it will have a bigger effect.

6 Discussion and limitations

7 Appendix - References

[1] https://mccyclingdiary.blogspot.com/2022/08/london-edinburg-london-lel-2022-edition.html

8 Appendix - Code

All analysis code is in `code.R` and is reproduced below.

```
# ============================================================
# Project 2: Modelling daily cycling demand in Edinburgh
# Authors: Alice Miller, Saba Sitter, Jake Saunders
# ============================================================

#
# Place the code needed in the Report_project02.Rmd, including documentation.
#
# 0. Packages
library(ggplot2); library(dplyr); library(tidyr)
library(lubridate); library(knitr); library(kableExtra)
library(patchwork); library(broom); library(gridExtra)

# 1. Load data
load('cycle_daily_df.Rdata')

# 2.1 Data Wrangling
cycle_daily_df <- cycle_daily_df %>%
  mutate(

    #Season indicator
    m_num = month(date),
    is_spring = ifelse(m_num %in% 3:5, 1, 0),
    is_summer = ifelse(m_num %in% 6:8, 1, 0),
    is_autumn = ifelse(m_num %in% 9:11, 1, 0),
    is_winter = ifelse(m_num %in% c(12, 1, 2), 1, 0),

    # Task 2: dow with explicit levels
    dow = factor(dow,
                 levels = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"),
                 ordered = TRUE),

    # Task 3: trend as integer days since 2020-01-01
    trend = as.numeric(date - date[1], units = "days"),

    #Covid indicator
    is_covid = ifelse(date >= "2020-03-23" & date <= "2021-03-31",1,0),

    #festive period indicator
    is_holiday = ifelse(m_num == 12 & day(date) >=24 & day(date)<=31, 1,0),

    #Daily temp range
    temp_range = temp_max - temp_min,

    #cold indicator
    is_freezing = ifelse(temp_min <= 0, 1, 0))%>%

  mutate(

    # Task 1: month as ordered factor for plots/inference
```

```r
    month = factor(m_num,
                   levels = 1:12,
                   labels = month.abb,
                   ordered = TRUE)

    )

# 2.2 exploratory plots

# Summary table for count, temp_mean, temp_min, temp_max
summary_stats <- cycle_daily_df %>%
  summarise(across(c(count, temp_mean, temp_min, temp_max),
                   list(Mean = ~mean(.x),
                        SD = ~sd(.x),
                        Min = ~min(.x),
                        Max = ~max(.x)),
                   .names = "{.col}.{.fn}")) %>% # use dots in variable names instead of underscore
  # Reshape for a vertical table
  pivot_longer(everything(),
               names_to = c("Variable", "Statistic"),
               names_sep = "\\.") %>% # Split at the dot instead of the underscore
  pivot_wider(names_from = Statistic, values_from = value)


# Time series of daily counts 2020-2025 with a smoother
plot_timeseries <- ggplot(cycle_daily_df, aes(x = date, y = count)) +
  geom_line(alpha = 0.4, color = "gray") +
  geom_smooth(method = "gam", color = "blue", se = TRUE) + # The smoother line
  scale_x_date(date_breaks = "12 months", date_labels = "%b %Y") +
  labs(
    title = "Daily Cycling Count in Edinburgh (2020-2025)",
    x = "Date",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

# Boxplots of count by month and by day of week.

# Boxplot by Month
plot_month <- ggplot(cycle_daily_df, aes(x = month, y = count)) + # Use fill = month to add colour
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Month", x = "Month", y = "Count") +
  theme_minimal()

# Boxplot by Day of Week
plot_dow <- ggplot(cycle_daily_df, aes(x = dow, y = count)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Day of Week", x = "Day", y = "Count") +
  theme_minimal()

# Scatter plot of count vs mean temperature
plot_temp_scatter <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count)) +
  geom_point(alpha = 0.3) + # alpha changes the opaqueness of the points
```

```r
  geom_smooth(method = "lm", color = "red", se = TRUE)+
  geom_smooth(method = "lm", formula = y ~ x + I(x^2), color = "blue") +
  labs(
    title = "Cycling Count vs. Mean Temperature",
    x = "Mean Daily Temperature (°C)",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

# non trivial plot - festive period
# Filter for December and calculate the average count for each day across all years
december_trends <- cycle_daily_df %>%
  filter(month(date) == 12) %>%
  mutate(day = day(date)) %>%
  group_by(day) %>%
  summarise(avg_count = mean(count, na.rm = TRUE))

# Create the plot
plot_december <-ggplot(december_trends, aes(x = day, y = avg_count)) +
  geom_line(linewidth = 1) +
  labs(title = "Average Cycling Demand in December",
       x = "Day in December",
       y = "Average Daily Cyclists") +
  theme_minimal()

# 3. Model Fitting
# Note: Use factor(month) in formulas for M1-M3
m0 <- lm(count ~ temp_mean + as.numeric(weekend) + as.numeric(month),
         data = cycle_daily_df)

##m1_literal has collinear terms so we define the factor versions of month and
#dow and omit the weekend.

m1 <- lm(count ~ temp_mean + trend + factor(month) + factor(dow),
         data = cycle_daily_df)

m2 <- lm(count ~ temp_mean + I(temp_mean^2)  + trend +
           factor(month) + factor(dow), data = cycle_daily_df )

m3 <- lm(log(count+1) ~ temp_mean + I(temp_mean^2) + trend +
               factor(month) + factor(dow)  + is_covid + is_holiday, data = cycle_daily_df)

# ggplots for models:

m0_fort <- fortify(m0)
m1_fort <- fortify(m1)
m2_fort <- fortify(m2)
m3_fort <- fortify(m3)


# Helper function to mimic Base R's plot(lm)
gg_diagnostic <- function(model_fort, type = 1, mod_name = "") {
  if (type == 1) { # Residuals vs Fitted
```

```r
    ggplot(model_fort, aes(.fitted, .resid)) +
      geom_point(alpha = 0.5) +
      geom_smooth(se = FALSE, color = "red", method = "loess") +
      geom_hline(yintercept = 0, linetype = "dashed") +
      labs(title = paste(mod_name, "Residuals vs Fitted"), x = "Fitted values", y = "Residuals")
  } else if (type == 2) { # Normal Q-Q
    ggplot(model_fort, aes(sample = .stdresid)) +
      stat_qq() + stat_qq_line(color = "red") +
      labs(title = paste(mod_name, "Normal Q-Q"), x = "Theoretical Quantiles", y = "Standardized Residua
  } else if (type == 3) { # Scale-Location
    ggplot(model_fort, aes(.fitted, sqrt(abs(.stdresid)))) +
      geom_point(alpha = 0.5) +
      geom_smooth(se = FALSE, color = "red", method = "loess") +
      labs(title = paste(mod_name, "Scale-Location"), x = "Fitted values", y = expression(sqrt("|Standa
  } else if (type == 4) { # Residuals vs Leverage
    ggplot(model_fort, aes(.hat, .stdresid)) +
      geom_point(alpha = 0.5) +
      geom_smooth(se = FALSE, color = "red", method = "loess") +
      geom_hline(yintercept = 0, linetype = "dashed") +
      labs(title = paste(mod_name, "Residuals vs Leverage"), x = "Leverage", y = "Standardized Residuals
  } + theme_bw()
}

p_m0_1 <- gg_diagnostic(m0_fort, 1, "m0")
p_m1_1 <- gg_diagnostic(m1_fort, 1, "m1")
p_m2_1 <- gg_diagnostic(m2_fort, 1, "m2")
p_m3_1 <- gg_diagnostic(m3_fort, 1, "m3")

p_m0_2 <- gg_diagnostic(m0_fort, 2, "m0")
p_m1_2 <- gg_diagnostic(m1_fort, 2, "m1")
p_m2_2 <- gg_diagnostic(m2_fort, 2, "m2")
p_m3_2 <- gg_diagnostic(m3_fort, 2, "m3")

p_m0_3 <- gg_diagnostic(m0_fort, 3, "m0")
p_m1_3 <- gg_diagnostic(m1_fort, 3, "m1")
p_m2_3 <- gg_diagnostic(m2_fort, 3, "m2")
p_m3_3<- gg_diagnostic(m3_fort, 3, "m3")

p_m0_4<- gg_diagnostic(m0_fort, 4, "m0")
p_m1_4 <- gg_diagnostic(m1_fort, 4, "m1")
p_m2_4 <- gg_diagnostic(m2_fort, 4, "m2")
p_m3_4<- gg_diagnostic(m3_fort, 4, "m3")

# 4. Cross-Validation Functions

# 4. Cross-Validation Functions

calc_scores <- function(y, mu, sigma, alpha = 0.05, model_name = "Model") {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 +
  # predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
```

```r
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  #RMSE
  RMSE <- sqrt(mean((y - mu)^2))

  #MAE
  MAE <- mean(abs(y - mu))

  #DS
  DS <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2))

  #IS
  lower <- mu - (qnorm(1-alpha/2) * sigma)
  upper <-mu + (qnorm(1-alpha/2) * sigma)
  IS <-  mean((upper - lower) +
               (2/alpha)* pmax(0,(lower - y)) +
               (2/alpha)*pmax(0,(y - upper)))

  # make a table with the results
  return (data.frame(Model = model_name, RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))
}




# 4. Cross-Validation Functions

calc_scores <- function(y, mu, sigma, alpha = 0.05, model_name = "Model") {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 +
  # predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  #RMSE
  RMSE <- sqrt(mean((y - mu)^2, na.rm = TRUE))

  #MAE
  MAE <- mean(abs(y - mu))

  #DS
  DS <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2), na.rm = TRUE)

  #IS
  lower <- mu - (qnorm(1-alpha/2) * sigma)
  upper <-mu + (qnorm(1-alpha/2) * sigma)
  IS <-  mean((upper - lower) +
```

```r
                (2/alpha)* pmax(0,(lower - y)) +
                (2/alpha)*pmax(0,(y - upper)))

  # make a table with the results
  return (data.frame(Model = model_name, RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))
}



# 5. Leave-One-Year-Out CV Loop
# Define models in a list ###change formulas!!!
models_list <- list(m0 = count ~temp_mean + as.numeric(weekend) +
                      as.numeric(month),
                    m1 = count~ temp_mean + I(temp_mean^2)  + trend +
                      factor(month) + factor(dow),
                    m2 = count ~ temp_mean + I(temp_mean^2) +
                      as.numeric(weekend) + trend + factor(month) + factor(dow),
                    m3 = log(count+1) ~ temp_mean + I(temp_mean^2) +
                      trend + factor(month) + factor(dow)  + is_covid  + is_holiday)

cycle_daily_df$date <- as.Date(cycle_daily_df$date)
cycle_daily_df$year <- year(cycle_daily_df$date)
years <- sort(unique(cycle_daily_df$year))
table1_results <- list()

for (y in years) {
  # Split: Extract one year for testing
  train_data <- subset(cycle_daily_df, year != y)
  test_data  <- subset(cycle_daily_df, year == y)

  for (mod_name in names(models_list)) {
    # Fit the model on the remaining years
    fit <- lm(models_list[[mod_name]], data = train_data)

    # Get predictive means (mu) and standard errors (se.fit)
    pred_obj <- predict(fit, newdata = test_data, se.fit = TRUE)
    mu <- pred_obj$fit

    # Calculate total predictive SD (sigma)
    # Combining residual variance and uncertainty in the mean
    res_std_error <- summary(fit)$sigma
    sigma <- sqrt(res_std_error^2 + pred_obj$se.fit^2)

    if (mod_name == "m3") { #transforming the log back
      mu_3 <- exp(mu)- 1
      sigma_3 <- mu_3 * res_std_error
      mu <- mu_3
      sigma<- sigma_3
    }


    # Run  calc_scores
    scores <- calc_scores(y = test_data$count, mu = mu, sigma = sigma, model_name = mod_name)
```

```r
    # Store result with identifiers
    table1_results[[paste(y, mod_name, sep="_")]] <- scores %>% mutate(Split_Year = y)

  }
}

#finding the average scores for each model and displaying
all_cv_results <- bind_rows(table1_results)
table1_final <- all_cv_results %>%
  group_by(Model) %>%
  summarise(across(c(RMSE, MAE, DS, IS), mean))

table1_final

#Table 2, CV RMSE and DS by month for final model

#fitting full models
m1_full <- lm(models_list$m1, data = cycle_daily_df)
m2_full <- lm(models_list$m2, data = cycle_daily_df)
m3_full <- lm(models_list$m3, data = cycle_daily_df)

mbest = m3_full

months_list <- month.abb
table2_results <- list()

for (m in months_list){

  #extract one month to test
  test_data <- subset(cycle_daily_df, month == m)

  # predict on the other month
  pred_obj <- predict(mbest, newdata = test_data, se.fit = TRUE)

  # Calculate total predictive SD (sigma)
  # Combining residual variance and uncertainty in the mean
  res_std_error <- summary(fit)$sigma

  #transform back
  mu_back <- exp(pred_obj$fit) - 1
  sigma_back <- mu_back * res_std_error

  # Run calc_scores
  scores <- calc_scores(y = test_data$count, mu = mu_back, sigma = sigma_back, model_name = "m3")

  # Store result with identifiers
  table2_results[[m]] <- data.frame(Month = m, RMSE = scores$RMSE, DS = scores$DS)
}

table2_final <- bind_rows(table2_results) %>%
  mutate(
    RMSE = round(RMSE, 1),
    DS = round(DS, 2)
```

```r
  )


#table 3 - trend coefficients from M1 - M3

#
# #getting the trend from each model
tr1 <- coef(m1_full)[["trend"]]
tr2 <- coef(m2_full)[["trend"]]
tr3 <- coef(m3_full)[["trend"]]
#
#building table 3
table3_final <- data.frame(
  #defining the rows as the 3 models
  Model = c("M1", "M2", "M3"),
  `Daily change (cyclists/day)` = c(tr1, tr2, tr3), #defining cyclists per day
  `Annual change (cyclists/year)` = c(tr1 * 365, tr2 * 365, tr3 * 365), #defining cyclists per year
  check.names = FALSE
) %>%
  mutate(
    `Daily change (cyclists/day)` = round(`Daily change (cyclists/day)`, 3), #rounding to 3
    `Annual change (cyclists/year)` = round(`Annual change (cyclists/year)`, 0) #rounding to 0
  )


# 5.1

# test and train
n = nrow(cycle_daily_df)
set.seed(1234)

idx <- sample(1:n, size = round(0.7 * n))

# Define the two datasets
train <- cycle_daily_df[idx, ]
test  <- cycle_daily_df[-idx, ]

M2_Tmean <-lm(count ~ temp_mean + I(temp_mean^2) + trend +

                factor(month) + factor(dow), data = train )

M2_Tmin <- lm(count ~ temp_mean + I(temp_min^2) + trend +

                factor(month) + factor(dow), data = train )

M2_Tmax <- lm(count ~ temp_mean + I(temp_max^2) + trend +

                factor(month) + factor(dow), data = train )

m2_scores <- function(model, data = test) {
  mu <- predict(model, data = test)
  se <- predict(model,data = test, se.fit = TRUE)$se.fit
  sigma <- sqrt(summary(model)$sigma^2 + se^2)
```

```r
  calc_scores(y = test$count, mu = mu, sigma = sigma)
}

# table
m2_comparison <- bind_rows(
  "Temp Mean" = m2_scores(M2_Tmean, test),
  "Temp Min"  = m2_scores(M2_Tmin, test),
  "Temp Max"  = m2_scores(M2_Tmax, test),
  .id = "Model")

# estimated marginal effect
# Extract the coefficients
# b1 is the linear slope, b2 is the curvature coefficient
b1 <- coef(M2_Tmax)["temp_mean"]
b2 <- coef(M2_Tmax)["I(temp_max^2)"]

# Calculate the Slope at 5C and 15C
# Formula: Slope = b1 + (2 * b2 * Temperature)
increase_5  <- b1 + (2 * b2 * 5)
increase_15 <- b1 + (2 * b2 * 15)

# 3. Round these for easy reading in your sentence
val_5  <- round(increase_5, 0)
val_15 <- round(increase_15, 0)

# Fit M2 with seasonal interactions
M2_interaction <- lm(count ~ temp_mean + (temp_mean : is_spring) + (temp_mean:is_winter) +
                       I(temp_max^2) + trend + factor(dow), data = train)

# Extract Coefficients
b_base   <- coef(M2_interaction)["temp_mean"]
b_spring <- coef(M2_interaction)["temp_mean:is_spring"]
b_winter <- coef(M2_interaction)["temp_mean:is_winter"]
b_quad   <- coef(M2_interaction)["I(temp_max^2)"]

# Calculate Marginal Effect at 5 degrees and 15 degress for both seasons
# Formula: (Base + Season_Bonus) + (2 * Curvature * Temp)
effect_spring_5 <- (b_base + b_spring) + (2 * b_quad * 5)
effect_spring_15 <- (b_base + b_spring) + (2 * b_quad * 15)
effect_winter_5 <- (b_base + b_winter) + (2 * b_quad * 5)
effect_winter_15 <- (b_base + b_winter) + (2 * b_quad * 15)
```